FutureSmart AI Blog

Follow

Integrating Google Authentication with FastAPI: A Step-by-Step Guide

Sakalya Mitra

Jan 16, 2025 · 25 min read

+2

15 1

Show more

In this blog, we'll walk through the process of enabling Google Authentication for a backend API system built using FastAPI. FastAPI has emerged as a popular choice for developing advanced GenAI applications and we at FutureSmart AI use it regularly to develop client applications. On top of FastAPI, Authentication is a critical component of any API-driven system as it ensures that only authorized users can access protected resources. By implementing authentication, you can safeguard sensitive data, prevent unauthorized access, and maintain the integrity of your services.

This implementation covers different authentication endpoints, using Google OAuth2 for authentication, and secure API access through google auth. Such a setup not only enhances security but also simplifies the user experience by leveraging Google's robust authentication system. Additionally, we'll add an endpoint that uses this authentication, demonstrating how authenticated APIs can provide tailored and secure responses.

Prerequisites

Before we begin, ensure you have the following ready:

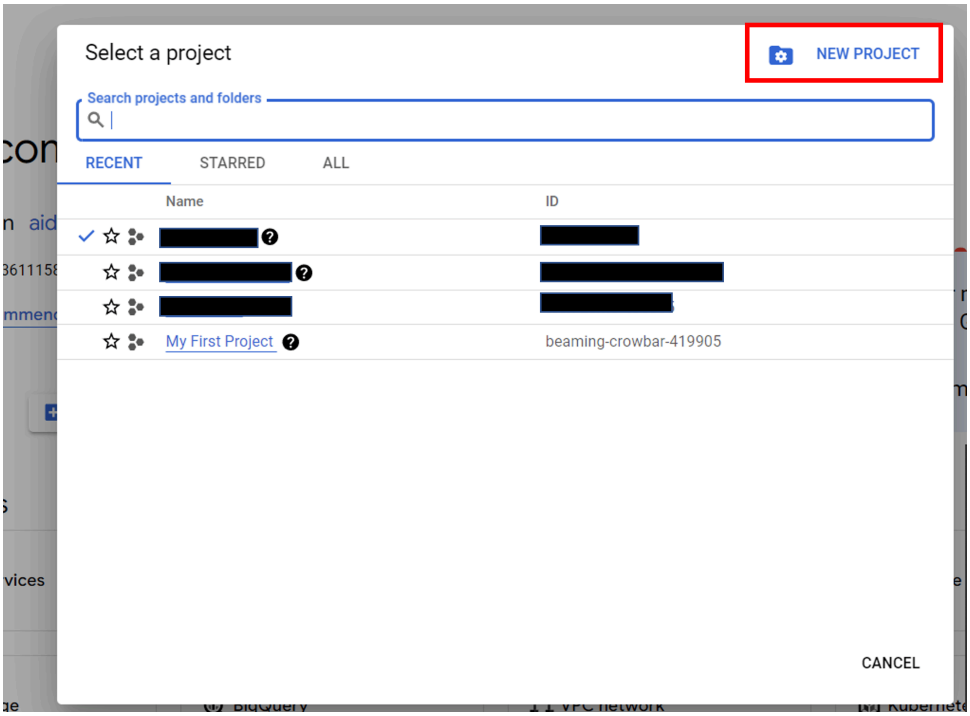
- **Google Cloud Console Account:** To set up the OAuth client credentials.
- **Python Environment:** Installed and configured with the required libraries

---

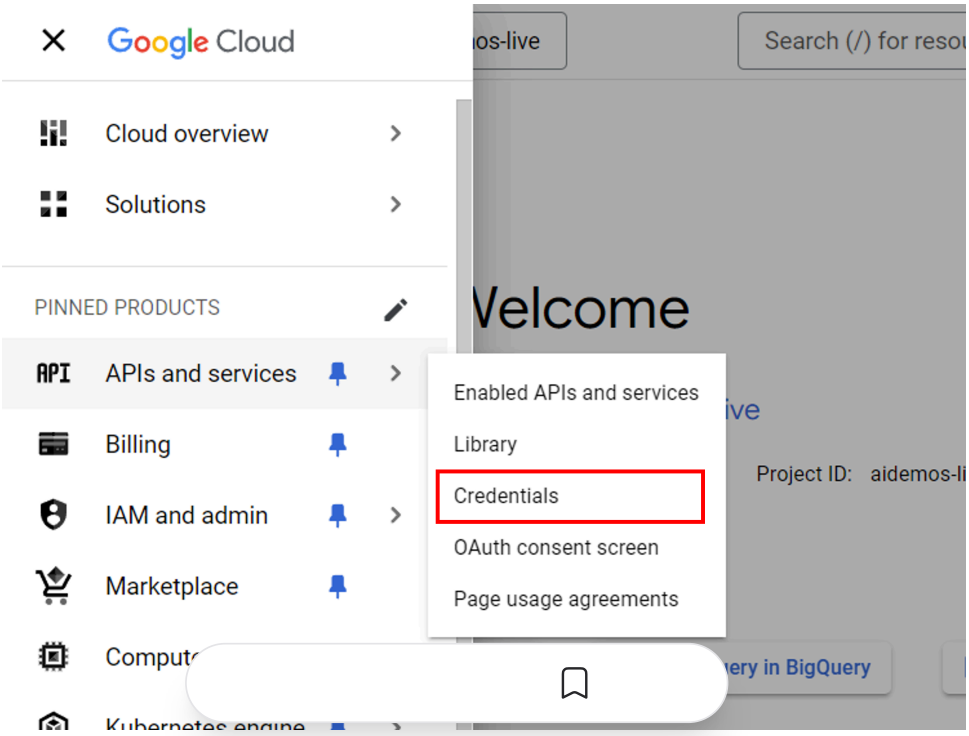
Step 1: Setting Up Google OAuth2 Client on Google Cloud Console

To enable Google a n OAuth2 client in the Google Cloud C

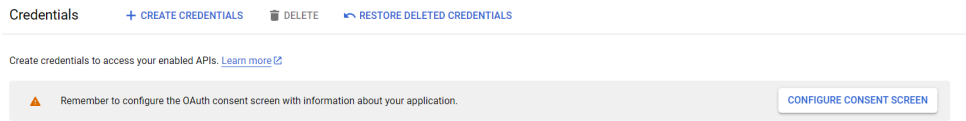
1. Navigate to the Google Cloud Console.
2. Create a new project or select an existing one.



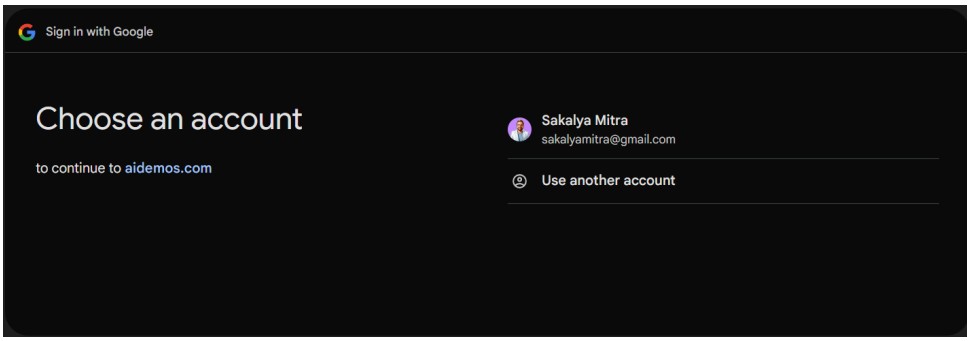
3. Once the project is opened/created, pull up the side bar and visit the **APIs and Services** section.
4. In that section select the "**Credentials**" options. The Credentials page will open.



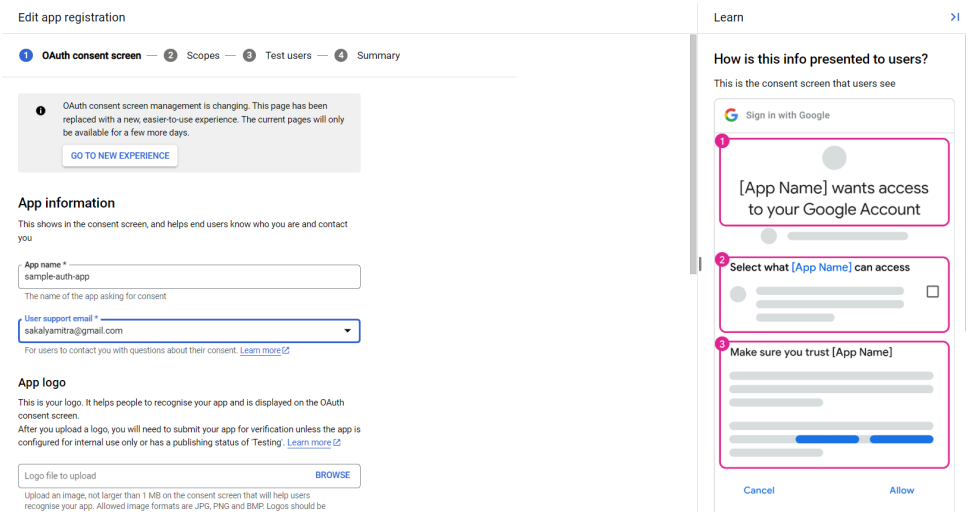
5. If this is a new project or your first project, you will be asked to configure the OAuth Consent Screen with your application details. This is a simple consent where the type of users who will be using your product is required (either External or Internal). Select it as per your requirement.



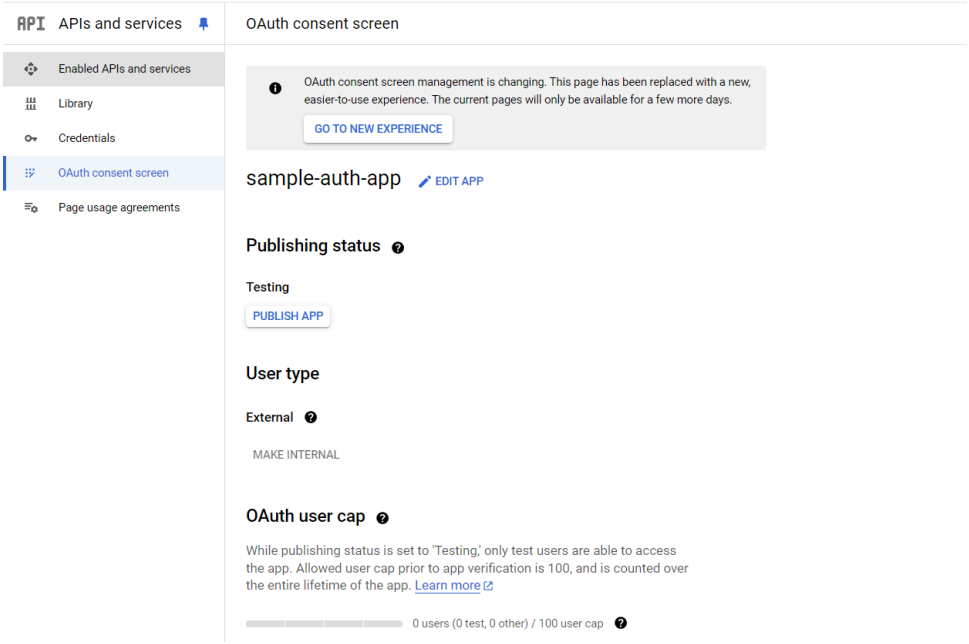
6. Once the previous step is completed, you will be prompted to configure the consent screen of your application. This is how the consent screen looks like for the AI Demos Playground and AI Demos Tools.



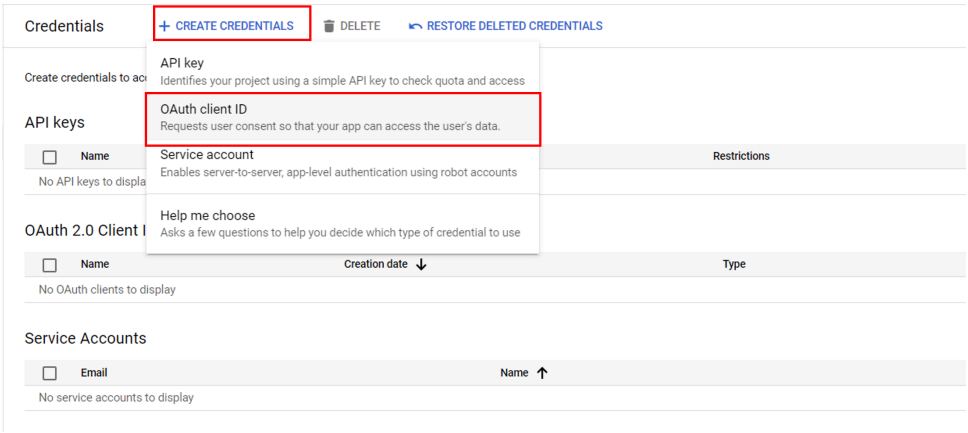
You can configure your application the way you want. You can put how the consent screen appears, what app name it shows, what logo to show and many more. Simply add your details and continue. For our ease we will add it as "*sample-auth-app*" and select myself as the developer. I will keep everything else as default and complete the setup.



This is how you… …e after the setup

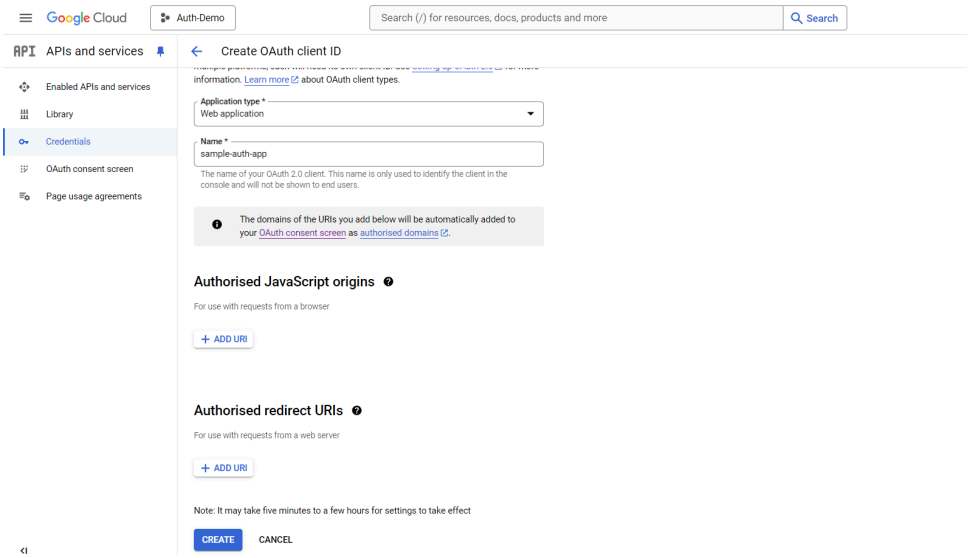


7. Now that you have the Consent screen configured, head over to the Credentials page and click on **“Create Credentials”.** Choose the **“OAuth Client ID”** option



This ensures that when someone tries to authenticate themselves to your app, it asks for the user consent and displays the consent screen with all the details for user to login.

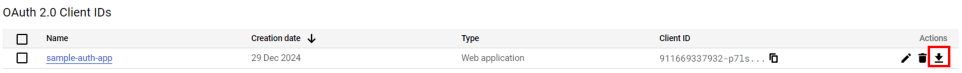
8. Once you are on the OAuth Client ID page, you will be prompted with two compulsory fields. One is the Application Type and other is the Name.



You can select the Application Type as per your app requirement. For this blog, we will select Web Application as we will be using it to authenticate our APIs via FastAPI. Also, I set the Name the same as our app: **sample-auth-app.**

The other two options named Authorised JavaScript Origins and Authorised redirect URIs are also something useful but we will configure them later in the blog with a clear understanding. For now we will keep them empty and click **"Create".**

9. We will have your Credentials created. We can now download the credentials as a secret JSON file from the home page.



We have successfully setup our OAuth Client and have completed the first step.

Step 2: Install Necessary Python Packages

We will install the necessary packages required to create the FastAPI endpoint and secure it with google auth.

COPY

```
python-dotenv
mysql-connector-p
requests
```

```
fastapi
uvicorn
python-jose[cryptography]
authlib
pyjwt
itsdangerous
google-auth
```

Step 3: Set up the Project Structure

We will follow a proper project structure for building our authenticated API system. Below is the sample project structure we will follow.

The main chat endpoint will be present in the `chat.py` file. The authentication setup will be present in the `auth.py` file. These will be routers and accessed via the main FastAPI app in the `api.py` file.

If you want to know in detail about how FastAPI works, you can check this detailed blog on Beginner's Guide to FastAPI and OpenAI Integration.

COPY

```
fastapi_auth_demo/
├── apis/
│   ├── chat.py
│   ├── auth.py
├── utils/
├── venv/
├── .env
├── .gitignore
├── api.py
├── client_secret.json
├── README.md
└── requirements.txt
```

Step 4: Configure Environment Variables

We will store sensitive credentials securely using environment variables. Create a `.env` file in the project directory and add:

COPY

```
GOOGLE_CLIENT_ID=<your-google-client-id>
GOOGLE_CLIENT_SECRET=<your-google-client-secret>
SECRET_KEY=<your-secret-key>
REDIRECT_URL=<your-redirect-url>
JWT_SECRET_KEY=<your-secret-key>
FRONTEND_URL=<your-frontend-url>
```

- You can directly get the `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` from the JSON file you downloaded in the first step.
- Keep the `REDIRECT_URL` and `FRONTEND_URL` empty for now and we will fill it with a proper value in the upcoming steps.
- The `SECRET_KEY` is a random string set by us, that is used to set the `state` parameter in the OAuth flow.
- The `JWT_SECRET_KEY` is a random string set by us, that is basically used as a key to encode and decode the access tokens that will be generated after authentication.

Step 5: Set Up Google OAuth Integration in FastAPI

The core functionality for Google authentication lies in integrating the OAuth2 flow into your FastAPI application. Below is the implementation:

Import Dependencies

Start by importing necessary modules and configuring OAuth:

COPY

```
# auth.py
from fastapi import FastAPI, Depends, HTTPException, status, Request, Cookie
from fastapi.responses import JSONResponse, RedirectResponse
from authlib.integrations.starlette_client import OAuth
from starlette.middleware.sessions import SessionMiddleware
from datetime import
from jose import
from dotenv import load_dotenv
import os
```

```
import uuid
import traceback

# Load environment variables
load_dotenv(override=True)

# App Configuration
app = FastAPI()
app.add_middleware(SessionMiddleware, secret_key=os.getenv("FASTAPI_SECRET_KE

# OAuth Setup
oauth = OAuth()
oauth.register(
    name="auth_demo",
    client_id=config("GOOGLE_CLIENT_ID"),
    client_secret=config("GOOGLE_CLIENT_SECRET"),
    authorize_url="https://accounts.google.com/o/oauth2/auth",
    authorize_params=None,
    access_token_url="https://accounts.google.com/o/oauth2/token",
    access_token_params=None,
    refresh_token_url=None,
    authorize_state=config("SECRET_KEY"),
    redirect_uri="http://127.0.0.1:8000/auth",
    jwks_uri="https://www.googleapis.com/oauth2/v3/certs",
    client_kwargs={"scope": "openid profile email"},
)

# JWT Configurations
SECRET_KEY = os.getenv("JWT_SECRET_KEY")
ALGORITHM = "HS256"
```

This what happens in this part of the code:

The `oauth = OAuth()` instance initializes the OAuth client. The `oauth.register()` method registers a new provider, in this case, Google. Here's what each parameter does:

- `name=”auth_demo”`
  - This is a unique name assigned to this OAuth2 provider configuration. It can be referenced in the application when invoking authentication flows.
- `client_id=confi`

- The `client_id` is fetched from configuration (usually environment variables). It identifies your application to Google's OAuth2 system.

- `client_secret=config("GOOGLE_CLIENT_SECRET")`
  - The `client_secret`, also fetched from configuration, is a secret key provided by Google that verifies your application during the OAuth2 flow. It should remain confidential.

- `authorize_url="https://accounts.google.com/o/oauth2/auth"`
  - This URL is the endpoint where users are redirected to log in with Google. It starts the OAuth2 authorization flow.

- `authorize_params=None`
  - Any additional parameters for the authorization URL can be specified here. Since it's `None`, no extra parameters are added.

- `access_token_url="https://accounts.google.com/o/oauth2/token"`
  - After the user logs in and grants permission, your application exchanges the authorization code for an access token at this endpoint.

- `access_token_params=None`
  - Specifies any additional parameters for obtaining the access token. Since it's `None`, no extra parameters are included.

- `refresh_token_url=None`
  - If your application needs to refresh tokens, you would specify the refresh token endpoint. Here it's `None`, indicating no token refresh functionality is configured.

- `authorize_state=config("SECRET_KEY")`
  - This optional parameter ensures the state parameter (used to prevent CSRF attacks) is securely managed. The `SECRET_KEY` provides added security.

- `redirect_uri="http://127.0.0.1:8000/auth"`
  - The `redirect_uri` is the URL to which Google redirects the user after a successful login. This must match the redirect URI configured in the Google Cloud Console. We will understand this in detail in the later part of the blog

- `jwks_uri="https://www.googleapis.com/oauth2/v3/certs"`
  - The **JWKS** keys needed to validate the tokens issued by Google.

- `client_kwargs={"scope": "openid profile email"}`
    - Defines the scopes (permissions) your application requests:
        - `openid` : Grants access to the user's identity.
        - `profile` : Provides access to the user's basic profile details, such as name and profile picture.
        - `email` : Grants access to the user's email address.

The second part of the code configures JWT, a token standard for securely transmitting information between parties.

1. `SECRET_KEY = os.getenv("JWT_SECRET_KEY")` :
    - Retrieves the secret key from the environment variables. This key is used to encode and decode JWT tokens.
    - It's essential to keep this key secure because anyone with access to it can forge valid tokens.
2. `ALGORITHM = "HS256"` :
    - Specifies the algorithm used for encoding and decoding JWT tokens. Here, **HS256** (HMAC using SHA-256) is chosen, which is widely used for its balance of security and performance.

Utility Functions for JWT

We'll use JWT (JSON Web Tokens) for managing user sessions securely:

COPY

```
def create_access_token(data: dict, expires_delta: timedelta = None):
    to_encode = data.copy()
    expire = datetime.utcnow() + (expires_delta or timedelta(minutes=30))
    to_encode.update({"exp": expire})
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)

def get_current_user(token: str = Cookie(None)):
    if not token:
        raise HTTPException(status_code=401, detail="Not authenticated")

    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        return {"user_id": payload.get("sub"), "email": payload.get("email")}
    except ExpiredSignatureError:
        raise HT                                                    expired")
```

```
    except JWTError:
        raise HTTPException(status_code=401, detail="Invalid token")
```

These are the two basic functions that is crucial.

1. `create_access_token` : This function takes the user data, and encodes it in a jwt token using the `SECRET_KEY` we have set in the .env file and the algorithm we have set. It also takes a parameter called expire which is the duration for which the access_token will be active. After this duration the access_token will expire and will not be a valid one to authenticate a user.
2. `get_current_user` : This function takes the access_token from the Cookie and decodes it to retrieve the information like user_id, email etc. Whenever an user goes through the login procedure, the access_token will be set as a browser cookie and can be directly accessed from there. We will see in the next part how this actually happens.

Login and Auth Endpoints

Create endpoints for logging in and authenticating users.

COPY

```
@router.get("/login")
async def login(request: Request):
    request.session.clear()
    referer = request.headers.get("referer")
    frontend_url = os.getenv("FRONTEND_URL")
    redirect_url = os.getenv("REDIRECT_URL")
    request.session["login_redirect"] = frontend_url

    return await oauth.auth_demo.authorize_redirect(request, redirect_url, pr
```

When we hit the endpoint `/login` , we will first fetch the referrer (the domain from where the endpoint is accessed or the request is received). Then we also fetch the `FRONTEND_URL` and `REDIRECT_URL` from the .env file. Now why are these required and what values to put in these variables is what we will discuss.

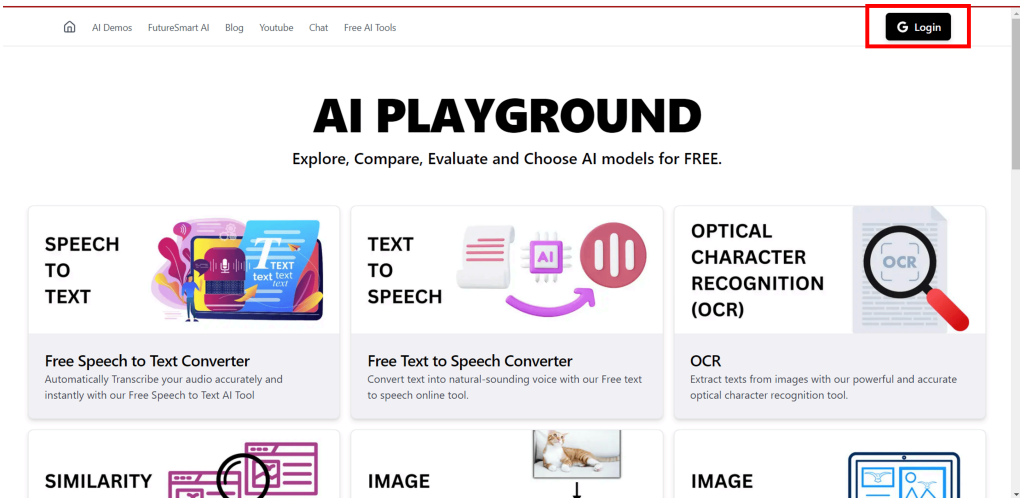
The google authentication process happens in the following way:

1. We send the re red OAuth Client. This ensures the ng in is shared with us and they consent for the same.

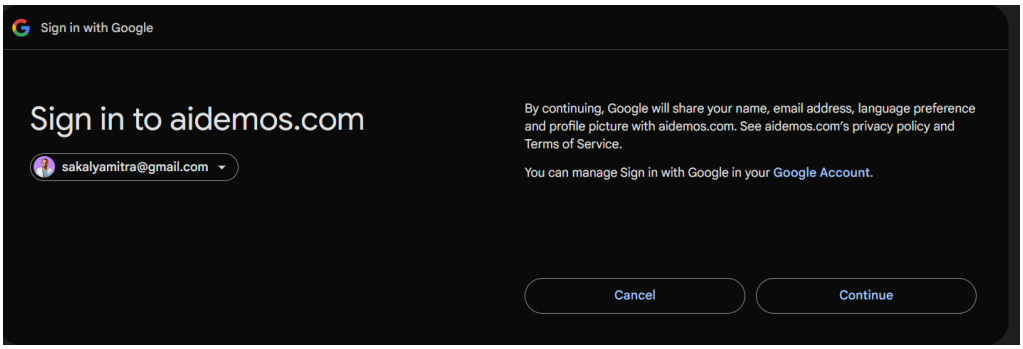
2. Once that is done and the user consents and allow to share information, Google allows the login and attaches the information in an access token.
3. The control now moves to our end to fetch the access token, decode it and retrieve the information about the user and verify it with our database or system. Now the redirect_url is responsible to decide in which URL should Google forward the control to for this verification and other process. In our case, we will use another router endpoint called /auth which will handle these requirements and we will set our auth endpoint as the redirect_url in the environment variable
4. Once this is done we can safely say the user is authenticated and allow them access to the apis.
5. The `FRONTEND_URL` parameter decides that once the user logs in, gets verified, where can we redirect them to.

Let's take the example of AI Demos Playground to understand this complete flow in a much better way.

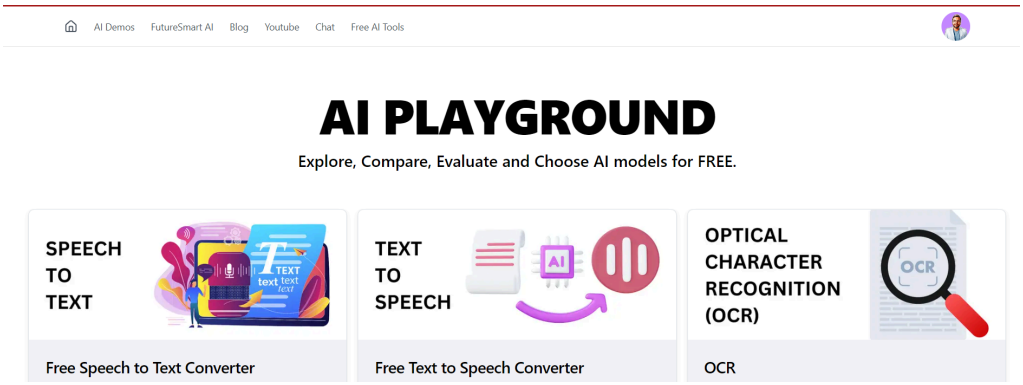
When you visit, AI Demos Playground you will see the Login button at the top right



When you click the Login button, you will be redirected to the Google Auth Consent Page to consent sharing of information with this website. What actually happens in the backend is that, when the Login button is clicked, it calls the /login api endpoint that redirects you to the google consent screen.



Now once you complete the process and consent, the control moves to the /auth endpoint that fetches the access token, verifies it, fetches information and stores them in Database. After all this the the user is redirected to the `FRONTEND_URL` which in this case will be the AI Demos Playground page from where you did Login.



Now that you have got the idea of the entire flow, let's move to the auth endpoint.

COPY

```
@router.route("/auth")
async def auth(request: Request):
    try:
        token = await oauth.auth_demo.authorize_access_token(request)
    except Exception as e:
        raise HTTPException(status_code=401, detail="Google authentication fa

    try:
        user_info_endpoint = "https://www.googleapis.com/oauth2/v2/userinfo"
        headers = {"Authorization": f'Bearer {token["access_token"]}'}
        google_response = requests.get(user_info_endpoint, headers=headers)
        user_info = google_response.json()
    except Exception as e:
        raise HTT                                    e authentication fa

    user = token.get("userinfo")
```

```
    expires_in = token.get("expires_in")
    user_id = user.get("sub")
    iss = user.get("iss")
    user_email = user.get("email")
    first_logged_in = datetime.utcnow()
    last_accessed = datetime.utcnow()

    user_name = user_info.get("name")
    user_pic = user_info.get("picture")

    if iss not in ["https://accounts.google.com", "accounts.google.com"]:
        raise HTTPException(status_code=401, detail="Google authentication fa

    if user_id is None:
        raise HTTPException(status_code=401, detail="Google authentication fa

    # Create JWT token
    access_token_expires = timedelta(seconds=expires_in)
    access_token = create_access_token(data={"sub": user_id, "email": user_em

    session_id = str(uuid.uuid4())
    log_user(user_id, user_email, user_name, user_pic, first_logged_in, last_
    log_token(access_token, user_email, session_id)

    redirect_url = request.session.pop("login_redirect", "")
    response = RedirectResponse(redirect_url)
    response.set_cookie(
        key="access_token",
        value=access_token,
        httponly=True,
        secure=True,  # Ensure you're using HTTPS
        samesite="strict",  # Set the SameSite attribute to None
    )

    return response
```

1. **Initial Token Exchange:**

```
token = await oauth.auth_demo.authorize_access_token(request)
```

COPY

- This exchanges [illegible] Google after user consent) for ar[illegible]

- It's the final step of the OAuth flow where Google confirms the user's authentication

2. **Getting Additional User Info:**

```
user_info_endpoint = "https://www.googleapis.com/oauth2/v2/userinfo"
headers = {"Authorization": f'Bearer {token["access_token"]}'}
google_response = requests.get(user_info_endpoint, headers=headers)
user_info = google_response.json()
```

- Makes a separate API call to Google to get more detailed user information
- Uses the access token to authorize this request
- Returns additional details like user's name and profile picture

3. **Extracting User Details:**

```
user = token.get("userinfo")
expires_in = token.get("expires_in")
user_id = user.get("sub")
iss = user.get("iss")
user_email = user.get("email")
user_name = user_info.get("name")
user_pic = user_info.get("picture")
```

- Extracts various user details from both the token and user info response
- `sub` is the unique Google user ID
- `iss` is the issuer (should be Google)
- Gets basic info like email, name, and profile picture

4. **Security Validation:**

```
if iss not in ["https://accounts.google.com", "accounts.google.com"]:
    raise HTTPException(status_code=401, detail="Google authentication failed
```

- Verifies that th
- Important security check to prevent token forgery

5. **Creating JWT Token:**

```
access_token_expires = timedelta(seconds=expires_in)
access_token = create_access_token(
    data={"sub": user_id, "email": user_email},
    expires_delta=access_token_expires
)
```

- Creates a JWT token containing the user's ID and email
- Uses the same expiration time as the Google token
- This JWT will be used for subsequent API calls

6. **Logging and Session Management:**

```
session_id = str(uuid.uuid4())
log_user(user_id, user_email, user_name, user_pic, first_logged_in, last_acce
log_token(access_token, user_email, session_id)
```

- Generates a unique session ID
- Logs the user's information and token for tracking/auditing
- Records when the user first logged in and last accessed the system

Here are the `log_token` and `log_user` functions which are simple Database CRUD operations

```
def log_user(user_id, user_email, user_name, user_pic, first_logged_in, last_
    try:
        connection = mysql.connector.connect(host=host, database=database, us

        if connection.is_connected():
            cursor = connection.cursor()
            sql_query = """SELECT COUNT(*) from users WHERE email_id = %s"""
            cursor.execute(sql_query, (user_email,))
            row_count = cursor.fetchone()[0]

            if ro
                s                                    email_id,user_name
                cursor.execute(sql_query, (user_id, user_email, user_name, us
```

```
            # Commit changes
            connection.commit()

    except Error as e:
        print("Error while connecting to MySQL", e)
    finally:
        if connection.is_connected():
            cursor.close()
            connection.close()

def log_token(access_token, user_email, session_id):
    try:
        connection = mysql.connector.connect(host=host, database=database, us

        if connection.is_connected():
            cursor = connection.cursor()

            # SQL query to insert data
            sql_query = """INSERT INTO issued_tokens (token, email_id, sessio
            # Execute the SQL query
            cursor.execute(sql_query, (access_token, user_email, session_id))

            # Commit changes
            connection.commit()

    except Error as e:
        print("Error while connecting to MySQL", e)
    finally:
        if connection.is_connected():
            cursor.close()
            connection.close()
            logger.info("MySQL connection is closed")
```

7. **Setting up Response:**

COPY

```
redirect_url = request.session.pop("login_redirect", "")
response = RedirectResponse(redirect_url)
response.set_cookie(
    key="access_token",
    value=access_token,
    httponly=True,
    secure=True,
```

```
        samesite="strict",
    )
```

- Gets the redirect URL that was stored before starting the OAuth flow
- Creates a response that will redirect the user back to the original page
- Sets the JWT token as an HTTP-only cookie with security flags:
    - `httponly` : Prevents JavaScript access to the cookie
    - `secure` : Cookie only sent over HTTPS
    - `samesite="strict"` : Prevents CSRF attacks by only sending cookie to same site

---

Now that we have understood the entire login and authentication process, it is now time to revisit some of the sections above that we have kept as empty.

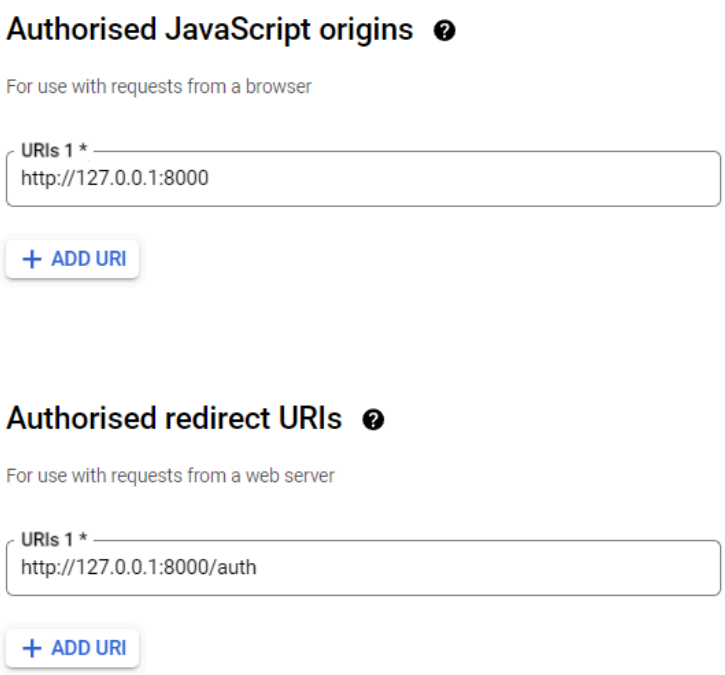
- While were registering the oauth client, we kept the redirect_uri parameter empty. We also kept the `REDIRECT_URL` parameter empty in the .env file. This will be assigned the url of our auth endpoint. The reason being after the login is successful from Google, it should again shift the control to the auth endpoint and we would be able to perform the necessary actions as discussed above. So it will be set to `127.0.0.1:8000/auth`
- We will be testing the login functionality via tha FastAPI Swagger Docs. So suppose we run the server on the 8000 port. It can be accessed at `http://127.0.0.1:8000/docs` . Now we want that after login and authentication we should be redirected to this page only. So we will set the FRONTEND_URL as `http://127.0.0.1:8000/docs` .
  This is how the final .env file look like

COPY

```
GOOGLE_CLIENT_ID=<your-google-client-id>
GOOGLE_CLIENT_SECRET=<your-google-client-secret>
REDIRECT_URL=http://127.0.0.1:8000/auth
JWT_SECRET_KEY="secret-key"
FRONTEND_URL=http://127.0.0.1:8000/docs
```

- In the Google C ty while creating our OAuth credentials. **Authorised JavaScript Origins** and **Authorised redirect URIs**.

- The **Authorised JavaScript Origins** is nothing but the source from which you are trying to login and authorize with the help of Google. In our case, we will request login from our localhost so it will be set to the same: `127.0.0.1:8000`
- The **Authorised redirect URIs** are the list of URLs that are authorised, to which Google can hand over control after the login process is done. As we want Google to shift control to the auth endpoint, we have to explicitly mention that it is a authorised redirect url from our end, so that Google doesn't block the request and allows it. If we don't specify this, the request after login will be blocked as Google ensures the access token and sensitive information is handed over to authorised uris only. This is how your Credentials configuration should look like.



Step 6: Create the API endpoint and Secure it with Google Auth Dependency

For securing the endpoint, we will be required to add a Dependency on the endpoint parameters. This dependency will be the access token which will only be available post login. Let's create the dependency first. We will create this dependency in the `auth.py` file.

COPY

```
def get_current_user(token: str = Cookie(None)):
    if not token:
        raise HTTPException(status_code=401, detail="Not authenticated")

    credentials_exception = HTTPException(
        status_code=401,
        detail="Could not validate credentials",
        headers={"WWW-Authenticate": "Bearer"},
    )
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])

        user_id: str = payload.get("sub")
        user_email: str = payload.get("email")

        if user_id is None or user_email is None:
            raise credentials_exception

        return {"user_id": user_id, "user_email": user_email}

    except ExpiredSignatureError:
        # Specifically handle expired tokens
        traceback.print_exc()
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail=
    except JWTError:
        # Handle other JWT-related errors
        traceback.print_exc()
        raise credentials_exception
    except Exception as e:
        traceback.print_exc()
        raise HTTPException(status_code=401, detail="Not Authenticated")

def validate_user_request(token: str = Cookie(None)):
    session_details = get_current_user(token)

    return session_details
```

We will now define the chat endpoint in the `chat.py` file.

COPY

```
from fastapi import Depends, APIRouter
from auth import get_current_user

router = APIRouter()
```

```
@router.get("/chat")
async def get_response(current_user: dict = Depends(get_current_user)):
    return {"message": "Welcome!", "user": current_user}
```

Step 7: Building the FastAPI App

Now that we have the authentication endpoint and chat endpoint ready, we can now setup our FastAPI app in the root of the project, in the `api.py` file.

COPY

```
import os
from fastapi import FastAPI, Header, HTTPException, Depends, Request
from starlette.config import Config
from fastapi.middleware.cors import CORSMiddleware
from starlette.middleware.sessions import SessionMiddleware
from apis import auth, chat
import time, requests

from dotenv import load_dotenv
load_dotenv(override=True)

config = Config(".env")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
    expose_headers=["*"]
)

# Add Session middleware
app.add_middleware(SessionMiddleware, secret_key=config("SECRET_KEY"))

# # Logging time taken for each api request
@app.middleware("http")
async def log_response_time(request: Request, call_next):
    start_time = time.time()
    response = await call_next(request)
    process_time
    logger.info(f                                    {process_time:.4f}
    return response
```

```
app.include_router(chat.router)
app.include_router(auth.router)

if __name__ == "__main__":
    import uvicorn
    import nest_asyncio
    nest_asyncio.apply()
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

Step 8: Running and Testing the Application

Run your application using Uvicorn:

COPY

```
uvicorn api:app --reload
```

1. **Login Endpoint:** Visit `/login` to authenticate with Google.
2. **Chat Endpoint:** Use `/chat` to see if you can see your details after authentication.
   If everything was done correctly, you will be able to see your FastAPI function the same way as the demo below.

https://youtu.be/630ZLpP6avA

If you were able to setup your Google Auth, integrate it with FastAPI and authenticate your endpoints, then kudos. You have completely understood the process and created a simple FastAPI application backed by Google Authentication working!!

Step 9: Deploying your Application

Now that we have successfully setup our FastAPI application locally and able to integrated Google [illegible] ad and deploy it on a VM. These APIs are the ones that will be consumed by the Frontend

Application. So they have to be accessible and hence deploying our application is a crucial step.

You can check out this video Deploy FastAPI & Open AI ChatGPT on AWS EC2 to understand how to deploy your FastAPI application on an AWS EC2 instance. It also has a beginner-friendly YouTube tutorial that can be followed to get the FastAPI application up and running on EC2 VM.

Now that we have our FastAPI application up and running on the VM (for this tutorial assume we have the app running on **8000 port**), it is time to move ahead and configure our domain and nginx that are some crucial steps in the completion of this backend deployment. As we are building this API to be consumed by the frontend, we have to make sure the frontend makes requests to our API and gets response.

Domain Mapping and SSL Certificate Generation

The first step in this configuration is to map our deployed endpoint to a domain and have a SSL certificate assigned to the same.

Why Our API Needs Domain Mapping and SSL

Modern browsers block direct IP access to protect users from potential security threats.

- Domain names provide a verifiable identity for your service. SSL certificates are issued to domains, not IPs, because domains can be validated through a chain of trust (Domain Registrar → Certificate Authority → Your Server). IPs can change hands frequently, making them unreliable for identity verification.
- Modern web browsers enforce HTTPS for security-critical features. They require:
  - A valid domain name (not an IP address)
  - An SSL certificate from a trusted authority
  - A match between the domain name and SSL certificate

Without these, browsers display warning messages and may block access entirely, especially for features like secure cookies, service workers, or HTTP/2 connections.

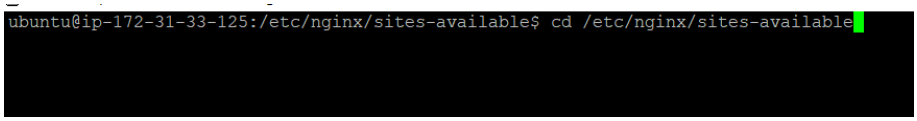
Consider accessing a bank's website. We trust `https://mybank.com` because:

- The domain ownership is verified
- The SSL certificate confirms you're connecting to the real bank
- Your data is encrypted during transmission

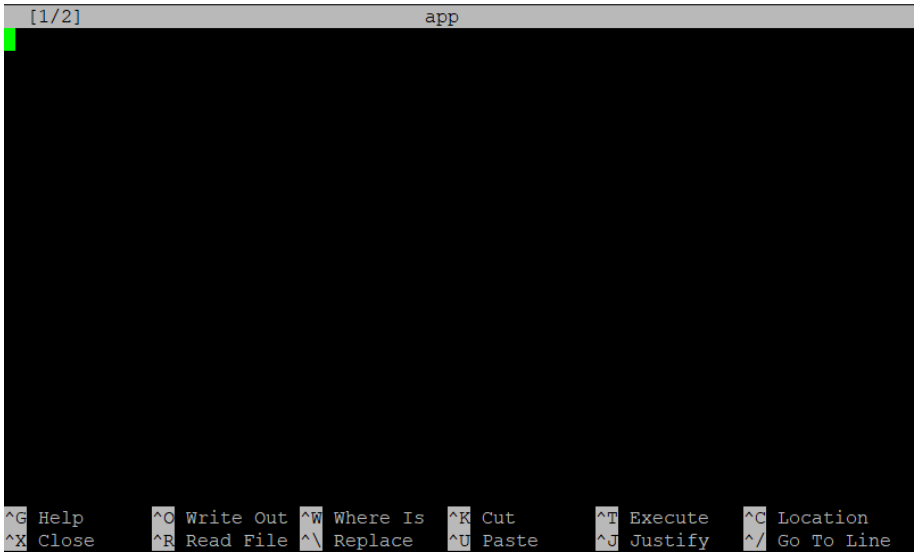
If the bank used just an IP (like `https://193.168.1.1`), we'd have no way to verify its authenticity, making it vulnerable to impersonation attacks.

Configuring this is very straightforward and fairly simple. We can follow these simple steps and we will be good to go:

1. The first step is to obtain a domain to which you will map your IP to. For example we already have a domain named, `sample-auth-app.futuresmartai.com` and this is mapped to the IP of our VM where the FastAPI application is hosted.
2. Now what we want is to ensure that our IP is associated with this domain name and any request coming to this domain, basically get transferred to our FastAPI application. To make this happen, we have to add some NGINX configuration in the VM where our FastAPI app is hosted.
    - Open your EC2 instance terminal and make sure to update and install nginx. You can run these two commands in the same order: `sudo apt update`, `sudo apt install nginx` . These make sure the libraries are updated and nginx is installed on your EC2 instance.
    - Move to the directory `/etc/nginx/sites-available`



    - Create a file for nginx configuration by running: `sudo nano appconfig` . This creates a file named appconfig and opens it in a VIM editor where we can directly put our configurations and save it.



- Now we can directly put our configuration in this file. Below is the NGINX config we will be using

COPY

```
server {
    listen 80;
    server_name sample-auth-app.futuresmartai.com;

    location / {
        proxy_pass http://127.0.0.1:8000; # Forward requests to the
        proxy_set_header Host $host; # Pass the original host heade
        proxy_set_header X-Real-IP $remote_addr; # Send the actual
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for
        proxy_set_header X-Forwarded-Proto $scheme; # Indicate the
    }
}
```

> 💡 These headers (Host, X-Real-IP, X-Forwarded-For) are critical for passing client IP addresses and cookies to the backend. Without them, backend logs will show the Nginx IP instead of the client's IP, and client cookies may not work correctly.

- We can put this in the appconfig file and save it using Ctrl + O and then press Enter. Finally press Ctrl + X to come out of the editor.
- We can now see the config in our file using, `cat appconfig`

```
ubuntu@ip-172-31-33-125:/etc/nginx/sites-available$ cat appconfig
server {
    listen 80;
    server_name sample-auth-app.futuresmartai.com;

    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
ubuntu@ip-172-31-33-125:/etc/nginx/sites-available$
```

- We now have to create a symbolic link using the nginx file in sites-available directory using: `sudo ln -s /etc/nginx/sites-available/appconfig /etc/nginx/sites-enabled/` . This is a best practice in Nginx server configuration because:
    1. 'sites-available' stores all our server configurations
    2. 'sites-enabled' only contains configurations that should be currently active
    3. The symbolic link allows to easily enable/disable sites without duplicating files
    4. When Nginx starts, it only reads configurations from 'sites-enabled'
- Now once this is done, we have to test if the nginx config file has no errors. Run: `sudo nginx -t`
- If the test is successful, reload the nginx configuration for the server to ensure these changes are applied using: `sudo systemctl restart nginx`

3. Thats it! Now we will be able to access our FastAPI application on the mapped domain: `sample-auth-app.futuresmart.ai/docs`

Common Mistakes

We have successfully built a FastAPI backend application using Google OAuth and deployed it to VM and mapped it to a certified domain name. When developing the frontend application that consumes these APIs, there are some common mistakes that tend to occur that can be avoided if the frontend and backend adjust these certain configurations:

Cross-Domain Cookie Access

This is one of the m many of our applications, we hav as a core mistake that needs to be taken care of during each application being developed.

As we have our FastAPI application hosted on an EC2 instance and mapped to domain. The Frontend application will similarly be deployed on an instance and mapped to a domain. But as the domain of the frontend and backend will be completely different, the cookie being shared becomes a problem. What I mean by this is that:

- When we are accessing the /login endpoint, we have seen that it verifies the identity via Google, creates an access token and sets it as a Cookie in the browser (more specifically in the domain).
- If the frontend sends a request to the /login endpoint, even after the entire authentication process, the token will be set as Cookie in the sample-auth-app.futuresmart.ai domain and not the domain where the frontend actually makes the request from
- The issue in this approach is that now the frontend has no access to the Cookie which is required to access the /chat endpoint and as well as get the user information

The solution to this is very simple and straightforward. We have to use a proxy from the frontend to ensure the Cookie is being set in the same domain from where the request is being sent.

- This has to be communicated to the frontend dev that on their end, they have to configure the nginx configuration such that the login request from the frontend is not directly made to the backend apis.
- They have to setup a proxy such that the request from frontend is sent to frontend domain/login endpoint and upon receiving that it should be sent to the actual backend api.
    - Let's understand this thing in a simple manner. Suppose we have our frontend at `sampe-auth-app-frontend.futuresmart.ai` . So any request from this domain has to be made via `sampe-auth-app-frontend.futuresmart.ai/login` . In the proxy configuration it has to be set such that the request coming to `sampe-auth-app-frontend.futuresmart.ai/login` has to be then redirected to the actual backend api which is `sampe-auth-app.futuresmart.ai/login` .
    - In the backend, the REDIRECT_URL in the .env file as well as the oauth object also has to be changed to `sampe-auth-app-frontend.futuresmart.ai/auth` . This is to ensure that when the login endpoint redirects its authenticated result, it again goes via the frontend and then r

- This will ensure that the communication always happens via the frontend and hence the Cookie will be set in the same domain from where the request is sent, which is `sampe-auth-app-frontend.futuresmart.ai`.

Cross-Origin Resource Sharing (CORS) Issue

This is another very common issue that we have faced in our projects. Mainly during testing locally, we tend to allow all origins to access the apis. But in production, we have to ensure we only allow the origins/domains that can access the apis explicitly. For example, in our case, we have to ensure that only the frontend domain can access our apis. So we have to set our CORS Middleware in the below way:

COPY

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["https://sampe-auth-app-frontend.futuresmart.ai"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
    expose_headers=["*"]
)
```

This ensures our application is secure and prevented from any random access.

Conclusion

In this tutorial, we've built a FastAPI application and integrated it with Google Authentication. The system showcases advanced concepts like JWT in FastAPI, OAuth 2.0 Implementation, Cookie based session and user management - which we have already implemented and successfully delivered in numerous enterprise solutions.

This architecture provides several advantages:

1. **Security**: Combines industry-standard JWT tokens, secure cookies, and Google's robust [illegible] layered security approach prote[illegible]ttack vectors.

2. **Maintainability**: Implements a clean, middleware-based architecture that centralizes authentication logic and makes it easy to protect new routes while keeping the codebase organized and manageable
3. **Scalability**: Leverages stateless JWT tokens and Google's infrastructure to create an authentication system that can easily scale across multiple servers and services without additional complexity.

If you found this guide helpful and want to explore more advanced backend development techniques, check out our other **tutorials**. If you want to know what skills truly matter to become a successful software developer or AI engineer, check out our blog on *Must-Have Skills for Upcoming Software Developers and AI Engineers in 2025*.

At FutureSmart AI, we specialize in building develop state-of-the-art AI solutions backed by scalable authentication systems and backend infrastructure for modern web applications. We've successfully implemented diverse authentication architectures across industries, from SaaS platforms to enterprise applications, while also specializing in AI integration by building numerous applications that incorporate Langchain, Langgraph, and ChatGPT through FastAPI frameworks.

For custom implementation support or consultations, feel free to reach out to us at contact@futuresmart.ai. For real-world examples of our work, visit our case studies where we showcase how our expertise in FastAPI, OAuth integration, and security best practices has helped businesses build robust authentication systems.

Stay tuned for our next tutorial in this series, where we'll dive into building the front-end for this system, accessing the API, and developing an end-to-end full-stack application!

Resources

💡 Get the Full Code in our GitHub

oauth FastAPI Cookie based authentication Google

google cloud

Written by

Sakalya Mitra

NLP Intern @FutureSmart AI | Former- Scaler, ITJobxs, Speakify | ML, DL Enthusiast| Researcher - Healthcare, AI

Follow

Published on

FutureSmart AI Blog

FutureSmart AI provides custom Natural Language Processing (NLP) solutions.

Follow

ARTICLE SERIES

Google Authentication: Integrating FastAPI, React, and Streamlit

1

Integrating Google Authentication with FastAPI: A Step-by-Step Guide

In this blog, we'll walk through the process of enabling Google Authentication for a backend API sys...